

High-Performance Computing

Prof. Dr. Ivan Kisel

Robin Lakos, Akhil Mithran, Oddharak Tyagi

High-Performance Computer Architectures

Practical Course

SoSe 2025

Issued: 25.06.2025 / Due: 02.07.2025



Institute for Computer Science

Exercise Sheet 9

Compute Unified Device Architecture (CUDA)

This exercise sheet focuses on Compute Unified Device Architecture (CUDA), a parallel computing platform and programming model developed by Nvidia. CUDA is a modern way of GPU programming for general-purpose applications. Many modern machine learning applications are developed with CUDA support for highly parallelized tensor calculations on GPU. It provides fine-grained control over GPU resources and offers powerful abstractions for parallelism, making it ideal for high-performance computing tasks that demand massive throughput and low-latency execution.

While CUDA offers deep integration with Nvidia hardware, it is inherently vendor-specific and requires a compatible Nvidia GPU. Setting up the CUDA environment involves ensuring the appropriate drivers and the CUDA toolkit are properly installed and configured, which may be challenging on certain systems.

On-site machines should already have CUDA installed and configured, allowing you to run the exercises without additional setup. If CUDA is not available or not functioning correctly on the machine you're using, please switch to another available machine on-site.

Submission: Submit your source code along with the PDF that includes your analysis.

Exercise 9.0:

After logging into a computer on-site, verify that the Nvidia GPU driver and CUDA is properly installed and working.

You can do this by running the command `nvidia-smi`. It should display information about the Nvidia GPU and driver status. If it fails or no GPU is shown, CUDA is probably also not running correctly on that machine.

Also make sure that the CUDA compiler NVCC is installed, e.g. by using `nvcc -version`.

If one of these fails, please switch to another computer (e.g. via SSH).

Exercise 9.1: Game of Life on GPU

In this exercise, you will extend your Game of Life project to leverage the GPU for the evolution steps using CUDA.

i) Preparation for CUDA

First, create two new files in your project:

- `src/World.cu`
- `include/World.cuh`

These files will be used for the CUDA specific source code.

As a next step, modify your `CMakeLists.txt`:

- enable CUDA support by adding the following to your `CMakeLists.txt` file:

```
project(yourProjectName LANGUAGES CXX CUDA)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CUDA_STANDARD 17)
set(CMAKE_CUDA_STANDARD_REQUIRED ON)
```

- add your new files to the appropriate target.

And finally, include the new CUDA header file where needed in your existing project code (e.g., where your CPU-based evolution function is located, to replace it later easily).

ii) Writing the Kernel

Write a CUDA kernel that updates the state of each cell in the Game of Life grid in parallel. Each thread should handle one cell or a small set of cells and determine its next state based on the current neighbors. Be sure to handle grid boundaries according to your implementation's rules (e.g., toroidal game world, etc.).

iii) Memory Management

Write a host function `EvoLveCUDA` that prepares and manages memory for the CUDA kernel execution. This function should be called instead of your CPU-based evolution function. Thus, it should expect the number of iterations that should be performed on GPU, the game's grid, and all necessary parameters. Provide these parameters as runtime and memory efficient as possible.

In this new function `EvoLveCUDA`, prepare everything required for the evolution steps on GPU

- Create CUDA streams – one non-default stream is sufficient.
- Allocate Memory – first, maybe start with unified memory to make the application running. Then, switch to manually managed memory. You do not have to submit the unified memory version.
- Launch Kernels – use an appropriate execution configuration.
- Synchronize – remember that you used streams.
- Clean up – deallocate memory and destroy stream(s).

In all relevant steps, make sure to check for errors!

iv) Performance Comparison

Compare the performance of your different evolution step implementations:

- Scalar (single-threaded CPU)
- OpenMP (multi-threaded CPU)
- CUDA (GPU)

Run each implementation on the provided `p67_snark_loop.txt` input for 500,000 iterations. Measure the total execution time using appropriate timing tools. Make sure that you do not print to console and neither have an active delay while benchmarking the different implementations.

Present the results in a table and calculate speedup factors relative to the scalar version.

Then, choose one of the following tasks:

- If available on the machine, use the CUDA profiler to analyze your algorithms performance. Then, copy the plot(s) from the profiler and discuss it in your PDF.
- Compare the algorithm execution runtime for different reasonable number of iterations. Then, plot and discuss the results in your PDF.